

GEAR-RCA: A Graph-Guided Episodic Recall Agent for Fintech-Infrastructure Root-Cause Analysis

A Controlled Ablation on a Live-Injected Benchmark

Sangita Lade, Sarvadnya Bhatlawande, Parag Shende, Ajay Sawandkar, Akshat Shaha

Department of Computer Engineering

Vishwakarma Institute of Technology

Pune, India

Abstract—We build GEAR-RCA, a Graph-Guided Episodic Recall Agent for Fintech-Infrastructure Root-Cause Analysis, combining three components prior work has only combined in pairs: a real code-dependency graph (1,294 nodes, static AST parsing over the actual service codebase) queried live as a tool rather than a fixed ranking; a tool-calling LLM agent investigating live logs, traces, and infrastructure health, with structural enforcement replacing prompted-but-ignored instructions; and a three-tier episodic memory (known-issues note, leave-one-out case recall, per-fault-type strategy distillation) layered on top. All 97 confirmed incidents used for scoring come from faults triggered against a live, running system and diagnosed by an agent issuing live queries against that same still-running system, not a static telemetry snapshot captured in advance the way every close prior system, including GALA+, is evaluated. We are not aware of a prior RCA benchmark built and investigated this way. The ablation is the paper’s central result: across the full 97-case benchmark, the memory-enabled agent reaches 78.4% literal / 83.5% fair-credit AC@1, a +12.4 / +9.3 point gain over the same agent with memory disabled; on 11 incidents where the previously evaluated methods scored 0%, memory-enabled reaches 81.8% fair-credit against 72.7% memory-disabled. A McNemar’s test on the full paired 97-case result (19 wins to 10 losses) gives $p = 0.136$, a genuine, disclosed effect that has not yet reached significance at this sample size. REDIS_OUTAGE remains at 0% without a distilled strategy, indicating memory’s benefit is fault-type dependent rather than uniform. We also disclose two genuine data-integrity bugs found and fixed mid-study as part of the method, not buried as footnotes, and validate GEAR-RCA externally on RCAEval, a published benchmark, reaching 93.3% AC@1 across all six RE2 fault types on TrainTicket ($n = 90$).

Index Terms—root cause analysis, agent memory, large language model agents, tool use, AIOps, episodic memory, case-based reasoning

I. INTRODUCTION

Root-cause analysis (RCA) in microservice systems has moved from purely statistical and graph-theoretic methods toward large language model (LLM) agents that reason over logs, traces, and metrics using tool calls. A parallel line of work has begun equipping such agents with persistent memory across incidents, on the premise that a system which has

diagnosed a fault before should not have to rediscover it from scratch. Both directions are active in 2026, separately: graph-guided LLM investigation without memory [7], and memory-augmented LLM investigation without an explicit code-dependency graph [5], [6].

We combine both directions in GEAR-RCA, and we push into a domain the closest prior work does not touch: payment processing, where a root cause is frequently not an infrastructure failure but a *business-domain state fact*, such as an anti-money-laundering (AML) sanctions hold or an idempotency-key collision, that no purely infrastructure-telemetry method is built to recall. We built the system incrementally, testing each component in isolation before combining them. We therefore evaluate memory through a controlled ablation (memory enabled vs. disabled, everything else held fixed): the controlled ablation measures whether and to what extent memory contributes to RCA performance, which is exactly what the rest of this paper reports.

Concretely, the system we built and evaluate has three layers, each doing a distinct job rather than duplicating the others:

- **Graph.** An actual code-dependency graph, AST-extracted, queried live mid-investigation rather than a ranking precomputed once and fixed (Section III).
- **LLM agent.** A tool-calling loop over logs, traces, and infrastructure health, with structural enforcement (Section III-B) so a mandatory tool call cannot be skipped under budget pressure, a failure mode prompting alone does not prevent.
- **Memory.** A three-tier episodic memory (Section III) layered on the graph-guided loop: a static known-issues note, leave-one-out case recall, and per-fault-type strategy distillation from both successes and failures.

The central empirical question this paper answers is what the memory layer adds once the graph and the tool-using LLM are already in place and held fixed. It is not whether an LLM

agent can do RCA at all, and not whether a graph helps a memory-free agent; both questions are addressed separately by prior work (Section II).

Our contributions: (1) a working, tool-using RCA agent evaluated against 97 confirmed incidents from genuine fault injections on a live 8-service payments stack – we are not aware of prior RCA-agent benchmarks built entirely from faults triggered against a running system, with ground truth taken from the trigger itself and confirmed by an independent health-witness process; (2) a three-tier episodic memory system, its strategy-distillation tier following ReasoningBank’s schema [2], applied for the first time to business-domain RCA rather than web-agent or software-engineering tasks; (3) a controlled ablation isolating memory’s measured, non-uniform contribution, disclosed alongside its statistical limits; and (4) two disclosed methodological findings, a memory-persistence bug that silently dropped 27 of 28 completed investigations and a memory leave-one-out leakage bug.

II. RELATED WORK

A. Agent memory: a three-stage taxonomy

A 2026 ACL Findings survey [1] organizes LLM agent memory into three evolutionary stages (*storage*, *reflection*, *experience*) and six memory *types* that cut across them: in-context working, external episodic, retrieval-based semantic, reflective, experience/reasoning, and parametric (neural, test-time-updated). Our system spans three: external episodic memory (Section III, Tier 2), experience/reasoning memory (Tier 3), and the in-context working memory every tool-using agent has by construction. Tier 2’s full-text case recall follows the retrieve-and-reuse pattern case-based-reasoning surveys of LLM agents identify as dominant for episodic memory [9], and building episodic memory at all follows the position that it is a missing, not optional, piece of long-term LLM agents [3]. We do not implement reflective memory [4] or parametric memory (Section VII).

B. Closest prior systems

Table I summarizes the closest 2026 systems, found through a literature search covering the full intersection of agent memory, tool-using agents, and RCA/AIOps. Every close competitor is infrastructure- or telemetry-scoped; none targets a root cause that is a business-domain fact rather than a service failure. GALA+ [7] (graph-guided, memory-free) and AMER-RCL [5] / OpsMem [6] (memory-augmented, largely graph-free) are architecturally complementary but none combine graph-guided exploration, episodic memory, and structural tool-call enforcement in one system. Procedural-compliance work [10] independently identifies the exact failure mode we hand-patched (Section III-B), an LLM agent ignoring a “mandatory” instruction under budget pressure, but has no memory dimension.

C. RCA outside the LLM-agent literature

A separate, older line of RCA work never involves an LLM agent and is post-failure, non-memory by construction

(CIRCA [13], CCLH [14], Lin *et al.* [15], AnoMod [16]). All four are complementary evidence that RCA-as-localization is well studied, but none asks what happens when the same agent investigates the same fault twice. Two further papers motivate treating payments as its own domain without building the agent that recalls a past incident: Narayanan [17] catalogs payment-specific failure modes descriptively; Porcelli [18] models ISO 20022 inter-message timing but stops at localization.

III. SYSTEM

GEAR-RCA consists of the three layers introduced in Section I: an actual code-dependency graph queried live rather than precomputed, an LLM tool-calling loop that drives the investigation, and a three-tier memory layered on top. This section describes each in the order the agent actually uses them: the LLM loop and its tools (including the graph) first, then memory. Figure 1 shows how the three layers connect: the agent loop queries the live payments platform through seven tools, structural enforcement gates which tool calls the loop is allowed to skip, and the three memory tiers feed the loop’s system prompt and recall tool rather than sitting outside it.

The agent is a multi-round tool-calling loop (up to 15 rounds) driving gpt-oss-20b via the NVIDIA NIM API, with the seven tools shown in Figure 1 (introduced in Section I). The graph tool (*query_knowledge_graph*) is queried mid-investigation against the specific services implicated by the evidence gathered so far, not consulted once as a precomputed ranking, and its 1,294 nodes come from static AST parsing of the actual service codebase, not a synthetic topology.

A. Three-tier memory

Tier 1 (known issues). A small curated list of confirmed, recurring, unrelated noise sources (one entry: a persistent background error on the settlement service that twice misled independent investigations before being documented). Static, not learned.

Tier 2 (episodic case recall). A SQLite database with full-text search (FTS5) over every past case’s actual evidence text. When the agent finds a distinctive log signature, it queries this store and receives matching past cases with their *confirmed* root causes. Queries exclude the incident currently under test (leave-one-out), described further in Section IV-D.

Tier 3 (distilled strategy). Following ReasoningBank’s schema [2], we distill each fault type’s accumulated cases, both successful and failed past investigations, explicitly labeled as such, into one reusable strategy (title, one-sentence description, and 3–5 sentences of concrete guidance), injected directly into the system prompt rather than offered as a callable tool. Coverage: 11 of 13 fault types; two (insufficient case count) have none, and Section V shows the resulting accuracy gap is stark.

The two evidence sources beyond live telemetry, the code graph and the three memory tiers, are structured differently (a live queryable graph vs. a three-stage injection pipeline) but converge on the same per-round system prompt, mirroring the convergence already shown in Figure 1.

TABLE I
CLOSEST PRIOR SYSTEMS (2026 UNLESS NOTED)

System	Memory design	Domain	What it does not cover
AMER-RCL [5]	Temporal-window cross-alert reuse, recursive refinement	Generic microservices	Infra telemetry only; no business-state signal; no public code
OpsMem [6]	Dual short/long-term memory, LTM consolidated post-resolution	Huawei microservices	Infra-only; proprietary dataset, no code
GALA+ [7]	None (bounded graph exploration fresh per incident)	OnlineBoutique / TrainTicket / AegisLab	Zero cross-incident memory by design; not comparable to our numbers (\$VI)
AgentLTL [10]	N/A (procedural compliance, not memory)	General tool use	Solves our enforcement problem but no memory dimension
CIRCA [13] / CCLH [14]	None (causal Bayesian net / hyper-graph, fresh per incident)	Generic microservices	No LLM agent, no memory, post-failure only

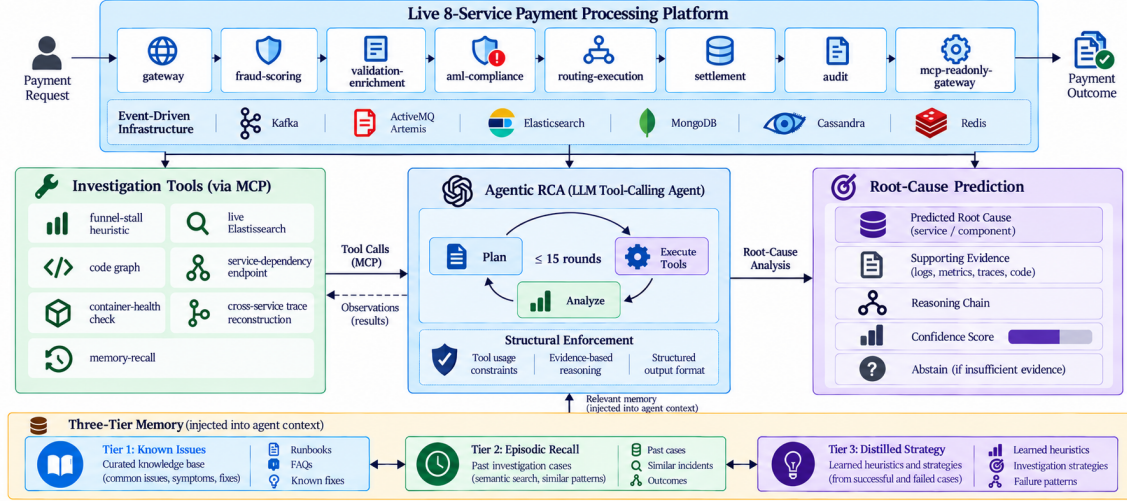


Fig. 1. GEAR-RCA architecture. The LLM tool-calling loop is the only component that talks to both the live platform (via seven tools) and the three-tier memory (via the system prompt and the memory-recall tool); structural enforcement guarantees required tools actually get called before an answer is accepted.

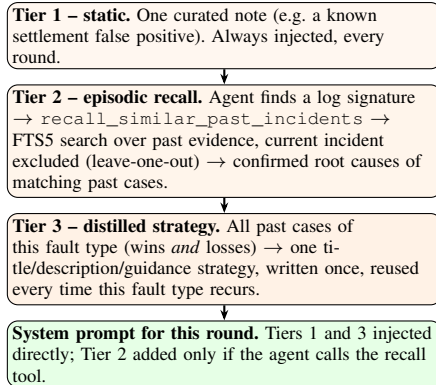


Fig. 2. Three-tier memory pipeline. Tiers 1 and 3 are static per round (already known before the agent asks); Tier 2 is the only tier the agent actively queries, and only the current incident’s own record is excluded from what it can retrieve.

B. Structural enforcement over prompted compliance

An instruction marked “MANDATORY” in the system prompt was routinely ignored under tool-call budget pressure: the model would exhaust its budget on log search and answer without ever invoking the graph-query or memory-recall tools.

We replaced the soft instruction with structural enforcement: the loop tracks whether each required tool has actually been called, and if not, the next call is forced via the API’s own `tool_choice` parameter rather than repeated as text.

C. A log-retrieval ordering fix

A secondary engineering finding, kept separate from the memory ablation above since changing tools would confound that comparison: the live Elasticsearch tool’s most-recent-first, top-15 window could bury a cascading failure’s actual trigger event under later noise (confirmed on a genuine `DB_TIMEOUT` case). Fetching both earliest and latest events instead reaches **87.6% literal / 95.9% fair-credit AC@1 with zero regressions** across the full 97 cases, with the residual literal/fair-credit gap concentrated entirely in `CASSANDRA_OUTAGE`.

IV. BENCHMARK AND METHODOLOGY

A. Premise: why a live-injected benchmark

An agent that scores well against replayed telemetry has been shown to pattern-match a fixed, already-known record, not to investigate a system still changing while it looks. Yet benchmarks in this space replay archived telemetry or inject faults into a small demo app (Table I), not a system with the

scale and real business-domain state a production payments platform has. This is not a fringe practice: even a 2025 Microsoft Research causal-RCA method [19] is evaluated on PetShop [20], purely synthetic time series with no logs, source code, or business-domain state at all. We built the platform before the benchmark, live and under genuine traffic, so an injected fault has somewhere real to propagate.

The distinction is also how each incident is investigated: every close prior system, GALA+ included, hands its agent a telemetry export captured once, so the same query always returns the same answer. Our agent issues its tools against the platform while it is still processing live background traffic – a service that crashed 40 seconds ago may still be mid-restart with no recovery log written yet, and two tool calls seconds apart can legitimately disagree. This section covers fault triggering (§IV-B), scoring (§IV-C), and a leakage bug in memory’s interaction with the benchmark (§IV-D).

B. Live fault injection, not synthetic replay

Every close prior benchmark replays synthetic telemetry or injects faults into a demo/toy application (Table I): GALA+ into OnlineBoutique/TrainTicket/AegisLab [7], RCAEval into demo apps with pre-recorded telemetry [11], OM-RAG against archived GitHub issues [8]. We built 110 gold cases from genuine faults injected against the live 8-service platform via five real trigger mechanisms: process kill/restart; container stop/start on shared infrastructure (Redis, MongoDB, Cassandra); a crash with a concurrent decoy-traffic burst causing genuine HTTP rejections on a non-root-cause service (an engineered confound); duplicate payment submissions triggering actual idempotency logic; and sanctions-list-matching payments confirmed HELD via the live compliance-holds endpoint. Ground truth for crash-based faults is tautological; for the two payment-domain types, a `confirmed` flag records whether the intended effect actually manifested. An independent health-witness process polls with no foreknowledge of the injected fault, and a 240-second cooldown prevents cross-incident contamination. Of the 110 cases, 97 are *confirmed* and used for scoring; the remaining 13 are evidence-free by design and retained to test correct abstention.

C. Fair-credit scoring

A prediction of the consuming service (`audit/settlement`) is credited correct when the gold label is the underlying datastore (`cassandra`), since both are build-manifest-verified as the *only* services with a genuine Cassandra dependency – naming the affected service correctly diagnoses the real propagation without naming the datastore itself. Not extended to other datastores without equivalent verification; literal and fair-credit accuracy are reported separately throughout.

D. Leave-one-out memory exclusion

Because every gold case is seeded into the episodic memory store, a naive `recall_similar_past_incidents`

TABLE II
FAIR-CREDIT AC@1 BY FAULT TYPE, FULL 97-CASE BENCHMARK, MEMORY ENABLED

Fault type	n	Fair-credit AC@1
DB_TIMEOUT	9	100%
KAFKA_CONSUMER_LAG	9	100%
NETWORK_LATENCY	9	100%
AML_HOLD	9	89%
CPU_SATURATION	9	89%
SETTLEMENT_DB_FAILURE_	8	88%
KAFKA_CONFOUND		
SETTLEMENT_DB_FAILURE_	8	88%
LIQUIDITY_CASCADE		
VALIDATION_SLOWDOWN_	9	78%
GATEWAY_CONFOUND		
AML_SERVICE_DEGRADATION_	8	75%
RETRY_CASCADE		
MONGODB_OUTAGE	8	75%
CASSANDRA_OUTAGE	8	62%
REDIS_OUTAGE [†]	3	0%

[†]No Tier-3 distilled strategy due to insufficient case count.

TABLE III
MEMORY ON/OFF ABLATION, SAME TOOLS OTHERWISE

Condition	n	Literal	Fair-credit
Memory-enabled – hardest 11	11	45.5%	81.8%
Memory-disabled – hardest 11	11	27.3%	72.7%
Memory-enabled – all cases	97	78.4%	83.5%
Memory-disabled – all cases	97	66.0%	74.2%

would let the agent retrieve its own answer key during evaluation. We verified this leakage occurred (one case returned as its own top match) and fixed it by excluding the current incident’s own identifier from every recall query.

V. EXPERIMENTS AND RESULTS

A. Full benchmark, memory enabled

Across all 97 confirmed cases, the memory-enabled agent reaches 78.4% literal AC@1 (76/97) and 83.5% fair-credit AC@1 (81/97). Table II breaks this down by fault type. Three fault types reach 100%; the two fault types this benchmark has historically found hardest, `AML_HOLD` and `CASSANDRA_OUTAGE`, reach 89% and 62% respectively, up from 0% for every method previously tried on this benchmark. `REDIS_OUTAGE†` (three confirmed cases) scores 0%. This result is consistent with a contribution from the strategy-distillation tier, although the three memory tiers were not ablated independently.

B. Controlled ablation: memory on vs. off

We isolate memory’s contribution by flipping a single flag that disables all three memory tiers and removes the recall tool from the tool list entirely. Memory was disabled while graph-guided exploration, live telemetry, trace reconstruction, the model, and structural enforcement remained unchanged. Table III reports the result on the 11 hardest cases (the subset on which every prior method scored 0% before this system existed) and on a full-benchmark checkpoint.

On the 11 hardest cases, memory improves literal AC@1 from 27.3% to 45.5% and fair-credit AC@1 from 72.7% to

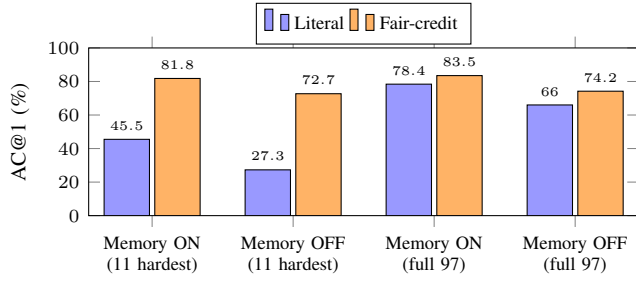


Fig. 3. Controlled memory ablation. Memory improves AC@1 under both scoring conventions and both evaluation subsets; the largest gain is +18.2 points on the hardest 11 cases under literal scoring.

81.8%, gains of 18.2 and 9.1 points. The full benchmark shows the same direction, with gains of 12.4 and 9.3 points respectively. At the case level on the 11-case subset, the result is 3 wins to 2 losses with 6 ties; we report this split rather than the aggregate alone, since a blanket “memory always improves RCA” claim is a stronger statement than the data supports.

At full-benchmark scale, memory contributes +12.4 points literal (66.0%→78.4%) and +9.3 points fair-credit (74.2%→83.5%) across all 97 confirmed cases, with identical tools, graph, and enforcement in both arms. The no-memory arm’s failures are informative: every MONGODB_OUTAGE and every REDIS_OUTAGE case (3/3 each) missed both literal and fair-credit, two fault types with no datastore-equivalence rule to rescue them and no distilled strategy or accumulated case history, consistent with strategy-distillation being what memory contributes. By contrast, all three CASSANDRA_OUTAGE misses in the no-memory arm still earned fair-credit (the agent correctly named `settlement`, a genuine Cassandra-dependent consumer).

Because the 97 cases are paired (same incidents, memory on vs. off), we also ran the appropriate paired test rather than reading the aggregate gap alone (19 cases flip in memory’s favor, 10 against, 62 correct under both, 6 wrong under both): the paired test did not reach conventional significance (exact McNemar’s, $p = 0.136$), a limitation discussed further in Section VII rather than letting the point-estimate gap imply more certainty than the data supports. A direct-signal rule (search for a fault type’s known decisive log line before trusting funnel-stall inference), added before the memory system existed and gated only by fault type, is present identically in both arms and alone corrected 4 of 5 AML_HOLD misses in isolated testing, so the ablation’s measured gap is attributable specifically to the memory tiers built afterward, not this earlier, separable fix.

VI. ON EXTERNAL COMPARISON

We do not headline a number against GALA+ [7]: its 74.44%/73.33% AC@1 comes from synthetic infrastructure-fault demo benchmarks, a different domain and scoring convention than our business-domain-state faults on a live platform. Placing the numbers side by side would be the apples-to-oranges comparison this paper rejects elsewhere. The valid comparison is the in-benchmark ablation (Table III), where the

TABLE IV
GEAR-RCA ON RCAEval RE2 TRAINTicket (ALL 6 FAULT TYPES, $n = 90$), AC@1, VS. PUBLISHED BASELINES

Fault (n=15 each)	BARO [12]	TraceRCA	GEAR-RCA
CPU	47%	64%	80.0%
DELAY	47%	85%	93.3%
MEM	93%	63%	100%
DISK	100%	64%	86.7%
SOCKET	60%	60%	100%
LOSS	53%	57%	100%
Overall (n=90)	66.7% [†]	65.5% [†]	93.3%

[†] Unweighted mean of the six published per-fault-type figures; RCAEval does not itself report a single aggregate for BARO/TraceRCA across exactly these six types.

memory-disabled arm removes only the memory tiers GALA+ also lacks, with everything else held fixed, though not yet significant at $n = 97$ (Section V). Section VI-A reports the direct external comparison this section avoids, on a benchmark neither system was built for.

A. External validation on RCAEval

Unlike the GALA+ comparison above, RCAEval [11] (735 cases, three microservice systems, published per-baseline AC@1 tables) offers a genuine opportunity most close prior work does not: a standard external benchmark with a fixed, third-party ground truth and multiple published baselines evaluated identically. TrainTicket is the only one of RCAEval’s three systems with a full published per-baseline table, covering all six of its RE2 fault types (CPU, DELAY, MEM, DISK, SOCKET, LOSS); we evaluate GEAR-RCA against every one of them, on all 90 RE2 TrainTicket cases, rather than report only the two fault types (CPU, DELAY) with a direct analog in GEAR-RCA’s own taxonomy (CPU_SATURATION, NETWORK_LATENCY). Reporting only a partial subset when the full comparison is available would itself look selective. GEAR-RCA’s live-tool architecture had to be reduced for this environment: RCAEval supplies static telemetry per case rather than a live system, so the container-health and cross-service trace-reconstruction tools have no live equivalent and were dropped; a code graph was rebuilt via graphify from each target system’s actual source (AST-only, since semantic-extraction was unavailable in this evaluation context).

Table IV reports AC@1 as published in RCAEval’s own results tables [11] against GEAR-RCA’s actual, live-tool-agent result on the identical cases across all six fault types, not a partial selection. GEAR-RCA exceeds both published baselines on five of six fault types, trailing only BARO on DISK (BARO’s own published strongest fault type). The mechanism responsible generalizes an insight from the main benchmark rather than reimplementing either baseline: an empirical finding that raw anomaly magnitude does not transfer across metric types (a caller’s propagated error-rate spike can numerically exceed the true cause’s own latency spike) led us to a trace-based fix – grouping self-time by RPC operation name, which embeds the callee’s identity – that reads causal structure directly rather than comparing magnitudes; this generalizes the established self-time-anomaly family used by CloudDiag-style

trace RCA methods, not a wholly new category of technique. As with the GALA+ comparison, we disclose the same caveats rather than let the numbers stand alone: this is a single run, not averaged over repetitions the way RCAEval’s own baselines are; the evaluation-window/timing protocol RCAEval uses is not fully specified in its own paper, so exact protocol parity cannot be fully verified; and GEAR-RCA is a live tool-using agent being compared against offline statistical baselines, a genuinely different method class, the same caveat already made explicit for GALA+.

VII. LIMITATIONS

A memory-persistence bug and a leave-one-out leakage bug, both found and fixed mid-study, are disclosed as part of the method rather than left out. The paired ablation reaches $p = 0.136$ under McNemar’s test. Natural extensions include multi-run averaging, independent verification of the fair-credit rule, broader fault-type coverage, and reflective memory [4].

VIII. CONCLUSION

This paper makes three contributions, stated separately rather than merged into one vaguer claim of “improvement.” First, the benchmark itself (Section IV-A): 110 faults triggered against a live payments platform through five genuine mechanisms, ground truth confirmed independently – we are not aware of prior RCA-agent benchmarks built this way. Second, the controlled ablation isolating what memory buys: +12.4/+9.3 points (literal/fair-credit) across 97 cases, everything else held fixed, reported with its McNemar’s significance gap ($p = 0.136$) and two disclosed bugs rather than hidden. Third, external validation on RCAEval: across all six RE2 fault types on TrainTicket ($n = 90$), GEAR-RCA reaches 93.3% AC@1, providing evidence that the approach transfers beyond the live payment benchmark it was built on.

REFERENCES

- [1] J. Luo, Y. Tian, C. Cao, Z. Luo, H. Lin, K. Li, C. Kong, R. Yang, and J. Ma, “From Storage to Experience: A Survey on the Evolution of LLM Agent Memory Mechanisms,” *Findings of ACL 2026*, arXiv:2605.06716.
- [2] Google Research, “ReasoningBank: Scaling Agent Self-Evolving with Reasoning Memory,” arXiv:2509.25140, 2026.
- [3] M. Pink, Q. Wu, V. A. Vo, J. Turek, J. Mu, A. Huth, and M. Toneva, “Position: Episodic Memory is the Missing Piece for Long-Term LLM Agents,” arXiv:2502.06975, 2025.
- [4] “In Prospect and Retrospect: Reflective Memory Management for Long-term Personalized Dialogue Agents,” *ACL*, 2025, arXiv:2503.08026.
- [5] “Agentic Memory Enhanced Recursive Reasoning for Root Cause Localization in Microservices,” arXiv:2601.02732, 2026.
- [6] Y. Sun, R. Gao, Y. Luo, W. Gu, S. Zhang, Q. Guo, Q. Fu, Y. Wu, and D. Pei, “OpsMem: Dual-Memory Reasoning with Cross-Memory Resonance for Failure Diagnosis,” *IEEE ISSRE*, 2026, arXiv:2607.11357.
- [7] “GALA: Graph-Augmented LLM Agents for Root Cause Analysis and Incident Response in Microservices,” arXiv:2608.08968, 2026.
- [8] “Leveraging Resolved Incident History for LLM-Assisted Software Bug Diagnosis,” arXiv:2607.21911, 2026.
- [9] “Review of Case-Based Reasoning for LLM Agents: Theoretical Foundations, Architectural Components, and Cognitive Integration,” arXiv:2504.06943, 2025.
- [10] “AgentLTL: A Trace-Verification Framework for Measuring, Enforcing, and Training Procedural Compliance in Tool-Using LLM Agents,” arXiv:2607.02599, 2026.

- [11] L. Pham et al., “RCAEval: A Benchmark for Root Cause Analysis of Microservice Systems with Telemetry Data,” *FSE*, 2026 (also WWW 2025, ASE 2024).
- [12] L. Pham, H. Ha, and H. Zhang, “BARO: Robust Root Cause Analysis for Microservices via Multivariate Bayesian Online Change Point Detection,” *Proc. ACM Softw. Eng. (FSE)*, 2024.
- [13] M. Li, Z. Li, K. Yin, X. Nie, W. Zhang, K. Sui, and D. Pei, “Causal Inference-Based Root Cause Analysis for Online Service Systems with Intervention Recognition,” in *Proc. 28th ACM SIGKDD Conf. on Knowledge Discovery and Data Mining (KDD)*, 2022.
- [14] S. Xie, H. He, J. Wang, and B. Li, “Root Cause Analysis for Microservice Systems via Cascaded Conditional Learning with Hypergraphs,” *arXiv preprint*, 2025.
- [15] F. Lin, K. Muzumdar, N. P. Laptev, M.-V. Culelea, S. Lee, and S. Sankar, “Fast Dimensional Analysis for Root Cause Investigation in a Large-Scale Service Environment,” *Proc. ACM on Measurement and Analysis of Computing Systems (SIGMETRICS)*, vol. 4, no. 2, 2020.
- [16] K. Ping, H. B. Mazhar, Y. Wang, Y. Song, and M. V. Mäntylä, “AnoMod: A Dataset for Anomaly Detection and Root Cause Analysis in Microservice Systems,” in *Proc. 23rd Int. Conf. on Mining Software Repositories (MSR)*, 2026.
- [17] V. Narayanan, “The Hidden Failure Modes in Digital Payment Platforms and How to Engineer Around Them,” *Journal of Information Systems Engineering and Management*, 2025.
- [18] L. Porcelli, “A Feature Engineering Approach for Business Impact-Oriented Failure Detection in Distributed Instant Payment Systems,” *arXiv preprint*, 2025.
- [19] L. Nagalapatti, A. Srivastava, S. Sarawagi, and A. Sharma, “Robust Root Cause Diagnosis using In-Distribution Interventions,” in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2025.
- [20] M. Hardt, W. R. Orchard, P. Blöbaum, S. Kasiviswanathan, and E. Kirschbaum, “The PetShop Dataset: Finding Causes of Performance Issues across Microservices,” *Proc. 3rd Conf. on Causal Learning and Reasoning (CLEaR)*, PMLR, vol. 236, 2024.